Python Data Structures

Lists

1. **Creating a list**:

```
fruits = ["apple", "banana", "cherry"]  # create a list of fruits
print(fruits)  # print the whole list
```

Output:

['apple', 'banana', 'cherry']

Homework:

Create a list of your favorite colors and print it.

2. **Accessing elements**:

```
fruits = ["apple", "banana", "cherry"]
print(fruits[1])  # print the second item (index 1)
```

Output:

Banana

Homework:

Given a list of three numbers, print the third number using indexing.

3. **Slicing a list**:

```
fruits = ["apple", "banana", "cherry", "date"]
print(fruits[1:3])  # slice from index 1 to 2 (includes banana, cherry)
```

Output:

['banana', 'cherry']

Homework:

Create a list of five animals and print the middle three using slicing.

4. **Negative indexing**:

```
fruits = ["apple", "banana", "cherry"]
print(fruits[-1])  # print the last item using negative index
```

Output:

cherry

Homework:

Given a list of four items, print the second to last item using negative indexing.

5. **Changing elements**:

```
fruits = ["apple", "banana", "cherry"]
fruits[0] = "mango"  # change the first item to 'mango'
print(fruits)
```

Output:

['mango', 'banana', 'cherry']

Homework: Change the last element of a list numbers = [10, 20, 30] to 40 and print the list.

6. **Adding elements with append**:

```
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")  # add 'orange' to the end of the list
print(fruits)
```

Output:

['apple', 'banana', 'cherry', 'orange']

Homework:

Start with numbers = [1, 2, 3]. Use append to add 4 and print the list.

7. **Inserting elements**:

```
fruits = ["apple", "banana", "cherry"]
fruits.insert(1, "kiwi")  # insert 'kiwi' at index 1
print(fruits)
```

Output:

['apple', 'kiwi', 'banana', 'cherry']

Homework:

Given letters = ['a', 'c', 'd'], insert 'b' at index 1 and print the list.

8. **Removing elements**:

```
fruits = ["apple", "banana", "cherry"]
fruits.remove("banana")  # remove 'banana' from the list
print(fruits)
```

Output:

['apple', 'cherry']

Homework:

Given nums = [1, 2, 3, 4], remove the number 3 and print the list.

9. **Looping through a list**:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)  # print each fruit on a new line
```

Output:

apple

banana

cherry

Homework:

Write a loop to print each item in the list colors = ['red', 'green', 'blue'] on its own line.

10. **List comprehension**:

```
nums = [1, 2, 3, 4]
squares = [x*x for x in nums]  # create a new list of squares
print(squares)
```

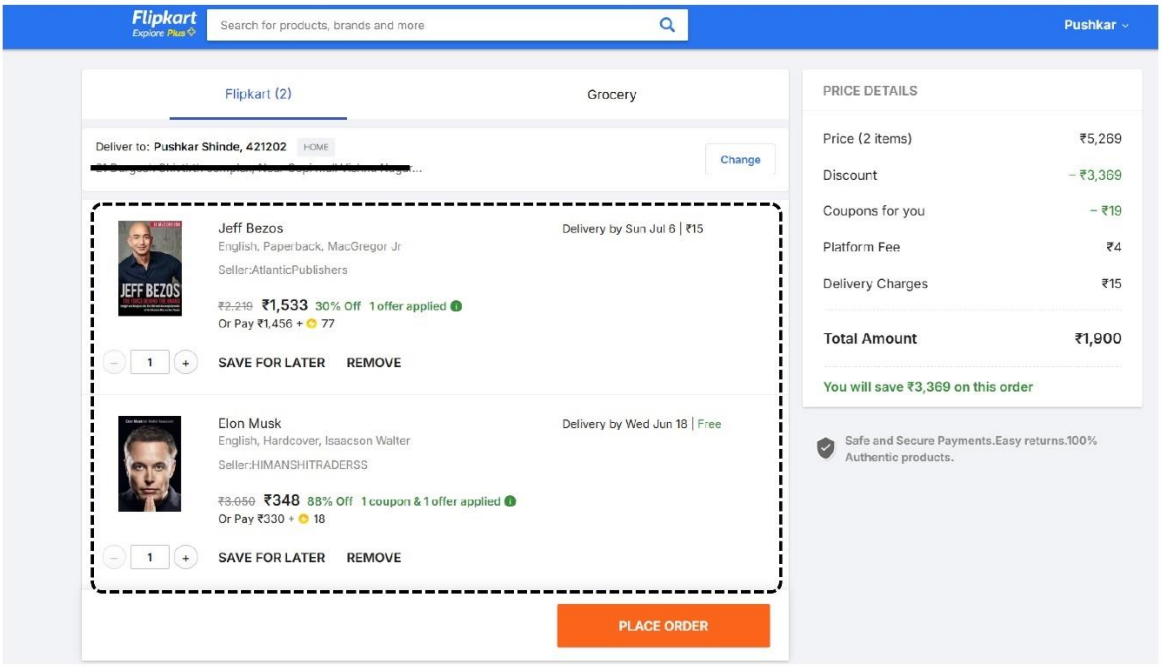
Output:

[1, 4, 9, 16]

Homework: Use a list comprehension to create a list of cubes (x^3) for x in [1, 2, 3, 4].

Practical Implementation:

- **Web and data applications:** Lists store things like items in a shopping cart or playlist. For example, cart = ['apple', 'milk', 'bread'] keeps cart items in order, making it easy to add or remove products.



Tuples

1. **Creating a tuple**:

```
coord = (10, 20)  # create a tuple of two numbers
print(coord)
```

Output:

(10, 20)

Homework:

Create a tuple with three elements (like (1, 2, 3)) and print it.

2. **Accessing elements**:

```
coord = (10, 20)
print(coord[0])  # print the first item (index 0)
```

Output:

10

Homework:

Given t = (5, 6, 7, 8), print the third element.

3. **Negative indexing**:

```
fruits = ("apple", "banana", "cherry")
print(fruits[-1])  # print the last item using negative index
```

Output:

Cherry

Homework:

Given a tuple of four colors, print the second to last color using negative indexing.

4. **Slicing a tuple**:

```
nums = (10, 20, 30, 40, 50)
print(nums[1:4])  # slice from index 1 to 3
```

Output:

(20, 30, 40)

Homework:

Slice the tuple (100, 200, 300, 400, 500) to get (200, 300).

5. **Unpacking a tuple**:

```
coord = (5, 7)
x, y = coord  # unpack tuple into two variables
print(x, y)
```

Output:

5 7

Homework:

Unpack the tuple info = ("Alice", 30, "NY") into three variables and print them.

6. **Counting elements**:

```
nums = (1, 2, 1, 3, 1)
print(nums.count(1))  # count how many times 1 appears
```

Output:

3

Homework:

Create a tuple (2, 3, 2, 2, 4) and print how many times 2 appears.

7. **Finding index**:

```
nums = (10, 20, 30, 20)
print(nums.index(20))  # find index of the first occurrence of 20
```

Output:

1

Homework:

For the tuple (4, 5, 6, 5, 7), find and print the index of the first 5.

8. **Concatenation**:

```
t1 = (1, 2, 3)
t2 = (4, 5)
t3 = t1 + t2  # join two tuples
print(t3)
```

Output:

(1, 2, 3, 4, 5)

Homework:

Join (7, 8) and (9, 10) into one tuple and print it.

9. **Single-element tuple**:

```
single = (5,)  # tuple with one element needs a comma
print(type(single))
```

Output:

<class 'tuple'>

Homework:

Create a tuple containing only the number 100 and print its type.

10. **Converting list to tuple**:

```
nums_list = [1, 2, 3]
nums_tuple = tuple(nums_list)  # convert list to tuple
print(nums_tuple)
```

Output:

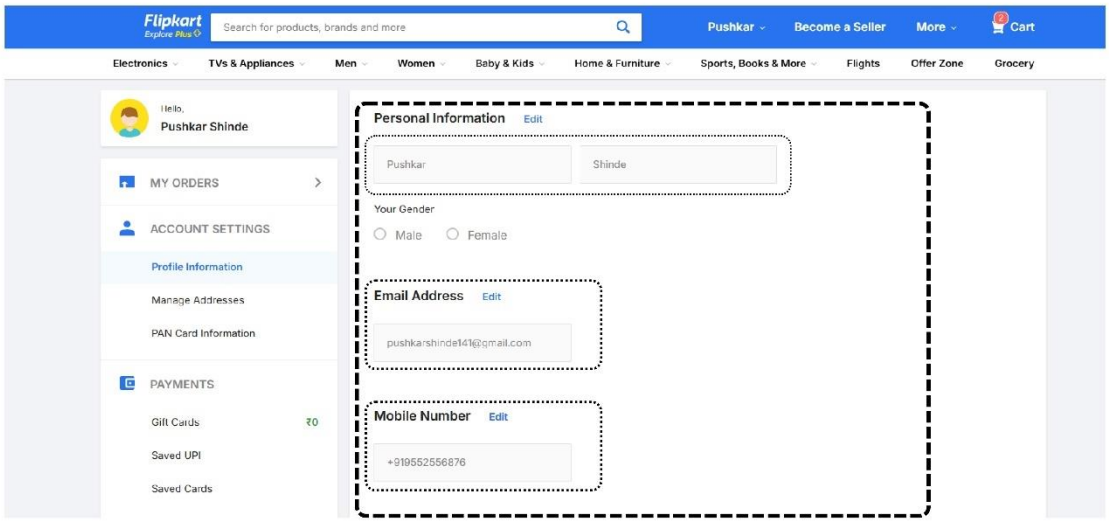
(1, 2, 3)

Homework:

Convert the list ['a', 'b', 'c'] into a tuple and print it.

Practical Implementation:

- **Fixed records:** Tuples hold data that shouldn't change. For example, a database record or a user profile might be (id, name, email), or a GPS location as (latitude, longitude). Since tuples are immutable, these values stay constant.



Dictionaries

1. **Creating a dictionary**:

```
person = {"name": "Alice", "age": 25}  # create a dict with string keys
print(person)
```

Output:

{'name': 'Alice', 'age': 25}

Homework:

Create a dictionary for a book with keys title, author, and year, then print it.

2. **Accessing values**:

```
person = {"name": "Alice", "age": 25}
print(person["name"])  # print the value for key 'name'
```

Output:

Alice

Homework:

Given student = {"id": 101, "grade": "A"}, print the student's grade.

3. **Adding key-value pairs**:

```
person = {"name": "Alice", "age": 25}
person["city"] = "NY"  # add a new key 'city'
print(person)
```

Output:

{'name': 'Alice', 'age': 25, 'city': 'NY'}

Homework:

Start with d = {"a": 1}. Add a new key "b" with value 2 and print d.

4. **Changing values**:

```
person = {"name": "Alice", "age": 25}
person["age"] = 26  # update the age to 26
print(person)
```

Output:

{'name': 'Alice', 'age': 26}

Homework:

Given d = {"x": 9, "y": 8}, change the value of "x" to 10 and print d.

5. **Removing key-value pairs**:

```
person = {"name": "Alice", "age": 25}
person.pop("age")  # remove the 'age' entry
print(person)
```

Output:

{'name': 'Alice'}

Homework:

Given d = {"a": 1, "b": 2}, remove key "a" using pop and print d.

6. **Keys and values**:

```
person = {"name": "Alice", "age": 25}
print(person.keys())   # keys in the dictionary
print(person.values()) # values in the dictionary
```

Output:

dict_keys(['name', 'age'])

dict_values(['Alice', 25])

Homework:

For d = {"a": 1, "b": 2}, print the list of keys and the list of values.

7. **Looping through a dictionary**:

```
person = {"name": "Alice", "age": 25}
for key, value in person.items():
    print(key, "->", value)  # print each key-value pair
```

Output:

name -> Alice

age -> 25

Homework:

Loop through d = {"x": 1, "y": 2, "z": 3} and print the key and value on each line.

8. **Checking key existence**:

```
person = {"name": "Alice", "age": 25}
print("name" in person)  # check if 'name' key is in dict
```

Output:

True

Homework:

Given d = {"a": 1, "b": 2}, check if the key "c" exists and print the result.

9. **Nested dictionary**:

```
person = {
"name": "Alice",
"info": {"age": 25, "city": "NY"}
}
print(person["info"]["city"])  # access nested dictionary
```

Output:

NY

Homework:

Create a nested dictionary d = {"x": {"y": 1}} and print the value 1.

10. Dictionary comprehension:

```
squares = {x: x*x for x in range(3)}  # keys and values from 0 to 2
print(squares)
```

Output:

{

0: 0,

1: 1,

2: 4

}

Homework:

Use a dictionary comprehension to map numbers 1 to 3 to their cubes (x: x^3) and print the dictionary.

Practical Implementation:

- **JSON and configurations:** Dictionaries map directly to JSON objects from APIs. For example, a web response {"name": "Pushkar", "age": 19} can be used as person = {"name": "Pushkar", "age": 19} in Python.

Sets

1. **Creating a set**:

```
nums = {1, 2, 3, 4}  # create a set of numbers
print(nums)
```

Output:

{1, 2, 3, 4}

Homework:

Create a set of three letters, e.g., {'a', 'b', 'c'}, and print it.

2. **Adding elements**:

```
nums = {1, 2, 3}
nums.add(4)  # add the number 4 to the set
print(nums)
```

Output:

{1, 2, 3, 4}

Homework:

Start with s = {'x', 'y'}. Add 'z' to the set and print it.

3. **Removing elements**:

```
nums = {1, 2, 3}
nums.remove(2)  # remove the element 2
print(nums)
```

Output:

{1, 3}

Homework:

Given s = {'a', 'b', 'c'}, remove 'b' and print the set.

4. **Set union**:

```
a = {1, 2}
b = {2, 3}
print(a.union(b))  # combine elements of a and b
```

Output:

{1, 2, 3}

Homework:

Given a = {10, 20} and b = {20, 30}, print the union of a and b.

5. **Set intersection**:

```
a = {1, 2}
b = {2, 3}
print(a.intersection(b))  # elements common to a and b
```

Output:

{2}

Homework:

Given a = {5, 6} and b = {6, 7}, print the intersection of a and b.

6. **Membership**:

```
nums = {1, 2, 3}
print(2 in nums)  # check if 2 is in the set
```

Output:

True

Homework:

Check if 'x' is in the set {'x', 'y', 'z'} and print the result.

7. **Looping through a set**:

```
nums = {1, 2, 3}
for x in nums:
    print(x)  # print each element
```

Output (order may vary):

1

2

3

Homework:

Write a loop to print each day in the set {'Mon', 'Tue', 'Wed'}.

8. **Set comprehension**:

```
squares = {x*x for x in range(5)}  # create set of squares
print(squares)
```

Output (order may vary):

{0, 1, 4, 9, 16}

Homework:

Use a set comprehension to create a set of even numbers from 0 to 8.

9. **Converting list to set**:

```
nums_list = [1, 2, 2, 3, 3]
unique_nums = set(nums_list)  # convert list to set (removes duplicates)
print(unique_nums)
```

Output (order may vary):

{1, 2, 3}

Homework:

Given lst = [4, 4, 5, 6, 6], create a set of unique values and print it.

10. **Set difference**:

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a - b)  # elements in a but not in b
```

Output:

{1}

Homework:

If a = {7, 8, 9} and b = {8, 10}, print the difference a - b.

Practical Implementation:

Track Available Positions: You can represent the game board as a set of available positions. When the player or bot places their move, you remove that position from the set.